ux_accessibility

UX-class (accessibility / usability) test coverage — BOB-110

Status: Coverage added. One real, currently-failing assertion documents a genuinely discovered defect in the live dashboard (see “Known finding” below) — it is deliberately left failing, not silenced, per §11.4.238 (automated QA must be the discoverer).

What was missing

`docs/testing/test_type_matrix.md`’s §11.4.27 test-type audit found UI **functional** coverage (`frontend/**/*.spec.ts` Vitest unit tests, `frontend/e2e/*.spec.ts` and `tests/e2e/*.py` Playwright walkthroughs) but **nothing framed around usability/ accessibility/UX outcomes specifically**. The one existing test with “accessible” in its name — `tests/integration/ test_ui_quick.py::test_dashboard_accessible` — asserts only `response.status_code == 200` and `"<app-root>" in r.text`; it is an HTTP-reachability check, not an accessibility check. This gap is exactly what BOB-110 (filed as a BOB-074 followup) scoped.

Suite location

```
tests/ux/
├── conftest.py                          # Playwright + axe-core
fixtures
├── fixtures/
│   ├── a11y_good.html                   # every a11y floor item
done right
│   ├── a11y_bad.html                    # same page, 4 planted
violations
│   ├── keyboard_good.html               # 6 controls, all keyboard-
reachable
│   └── keyboard_bad.html                # unreachable control +
```

```
invisible focus
├── test_accessibility_axe.py          # axe-core oracle
validation (fixtures)
├── test_keyboard_navigation.py        # real Tab-key traversal
(fixtures)
└── test_live_dashboard_accessibility.py  # same oracle vs the
REAL product
```

`frontend/package.json` gained one new devDependency: `axe-core` (installed via `npm install --save-dev axe-core`, real npm registry install, not vendored by hand) so the oracle's JS is available at `frontend/node_modules/axe-core/axe.min.js`.

Surfaces that actually exist (verified, not assumed)

Two candidate UI surfaces were checked before writing anything:

1. **`frontend/`** — Angular 21 dashboard, Vitest unit tests, Playwright e2e skeleton. The e2e skeleton (`frontend/e2e/README.md`) requires a manually started `ng serve` dev server on `:4200`, which was **not** running in this session (`curl http://localhost:4200/` → connection refused). Not used as the live target for that reason — starting a second dev server was out of scope/resource-budget for this dispatch.
2. **The merge-service root at `http://localhost:7187/`** — confirmed live (`curl` → `200`). Its raw HTML is `<app-root></app-root>` (the compiled Angular bundle, same app as `frontend/`, served statically) — **not** the separate Jinja2 `download-proxy/src/ui/templates/dashboard.html`, which is not what answers `GET /` today. This was verified, not assumed: `curl -s http://localhost:7187/` shows `<script src="main-VUXUAYF5.js">` and `<base href="/">`, i.e. the Angular SPA shell.

`test_live_dashboard_accessibility.py` targets surface (2) via the project's standard `merge_service_live` pytest fixture (`tests/fixtures/services.py`, the same fixture every other integration/e2e test in this repo uses) so it inherits the existing topology-detection / honest-skip behaviour.

Oracle chosen, and why the weaker ones were rejected

Chosen: a real axe-core scan of a real, Playwright-rendered, JS-hydrated DOM. Descending strength, per §11.4.170 (value-equality assertions are forbidden as the *proof* a UI is correct):

Candidate oracle	Verdict	Reason
Real axe-core scan of rendered DOM	**Used**	Implements the actual WCAG 2.x success criteria (real computed contrast, real ARIA resolution, real accessible-name computation) against a real browser's render of the real page.
Rendered-DOM assertions without axe (hand-rolled)	Used *in addition*, for structural facts axe does not enforce (single `<h1>`, real keyboard Tab order, real computed focus style)	Still a real-DOM oracle, just axe doesn't have a rule for these specific structural facts.
Static template grep (`grep alt= *.component.html`)	**Rejected as primary oracle**	Cannot see computed contrast, cannot see ARIA state Angular sets at runtime, and — concretely proven in this session — the merge-service root ships an **empty** `<app-root></app-root>` until JS executes (`curl` confirms this). A grep-only oracle would be auditing a page nobody ever sees; it is also exactly the oracle-agrees-with-source pattern that would pass a component that never renders.
Value-equality (e.g. asserting a hex string == an expected hex string)	**Forbidden outright (§11.4.170)**	Never used anywhere in this suite.

Accessibility floor covered (each assertion can genuinely FAIL)

Requirement	How it is asserted	Real fail path proven
Images have alt text	axe `image-alt` rule	`a11y_bad.html`'s alt-less `<img>` is flagged; verified by rule id + exact offending `<img>` HTML snippet.
Form controls have associated labels	axe `label` rule	`a11y_bad.html`'s disassociated `<span>Search query</span>` + unlabeled `#q` input is flagged.
Interactive controls are keyboard-reachable	Real `page.keyboard.press("Tab")` traversal, asserting on `document.activeElement.id` after each press	`keyboard_bad.html`'s `tabindex="-1"` button is proven never reached across 8 real Tab presses, while still `is_visible()` (so "not reached" means keyboard-unreachable, not "doesn't exist").
Interactive controls have `:focus-visible`	Real computed `outlineStyle`/`outlineWidth`/`boxShadow` read **after** a real Tab press	`keyboard_bad.html`'s `#inp1:focus{outline:none;box-shadow:none;}` is proven to produce no visible indicator.
Colour contrast meets WCAG AA (≥ 4.5:1)	axe `color-contrast` rule (computes real rendered contrast)	`a11y_bad.html`'s `#aaaaaa` on `#ffffff` (2.32:1, computed independently in Python against the WCAG relative-luminance formula before authoring the fixture) is flagged. **Also caught the real live-dashboard defect below — this is not just a fixture demonstration.**
One `<h1>`, sane heading outline	`page.locator("h1").count()` (real DOM) + axe `heading-order` (in the default WCAG tag set)	Direct DOM count assertion; `a11y_good.html` has exactly one, and the count is a real post-render count, not a hardcoded expectation about markup that was never rendered.
`<html lang>` present	axe `html-has-lang` rule + `document.documentElement.lang` read	`a11y_bad.html` omits `<html lang>` entirely; both the axe rule and the direct DOM read confirm it.

Golden-FALSE cases (correct patterns that must NOT be flagged)

Both are present in **`a11y_bad.html`** — the same page that has four real, flagged violations — specifically so the proof is that the oracle discriminates pattern-by-pattern, not merely “the whole page happened to pass”:

1. **Decorative image, `alt=""`** (`test_decorative_image_alt_empty_not_flagged`) — asserted absent from every violation’s node list.
2. **`aria-hidden="true"` decorative icon inside an `aria-label`-carrying button** (`test_aria_hidden_icon_not_flagged`) — asserted absent from every violation’s node list, plus an explicit check that `button-name` / `aria-hidden-body` never fire for the button itself.

RED → GREEN evidence (this session)

`tests/ux/fixtures/a11y_good.html`’s meaningful `<img>` alt attribute was removed, the golden-good suite was run and observed to **FAIL** on `image-alt`, the file was restored, and a `sha256sum` before/after proved a byte-identical restore before re-running to observe **PASS** again. Full pasted transcript in the BOB-110 dispatch report (agent turn, this session) — summary:

```
sha256 before:
15d83ce66c33889a1c876505e3892f979800123e2d9b25e3c504efac96e7d47e
RED:            TestGoldenGoodFixture::test_zero_wcag_violations
FAILED
                (image-alt: "Images must have alternative text")
sha256 after restore:
15d83ce66c33889a1c876505e3892f979800123e2d9b25e3c504efac96e7d47e
(IDENTICAL)
GREEN:          TestGoldenGoodFixture::test_zero_wcag_violations
PASSED
```

Known finding — real, currently-failing, deliberately not silenced

`test_live_dashboard_accessibility.py::test_live_dashboard_axe_scan` **fails today** against the real live dashboard. This is a genuine discovered defect, not a fixture artefact and not a test bug — axe reports `color-contrast` (serious) on 21 nodes, including the header brand text, the page `<h1>`, and the tagline `<p>`. Exact computed colours + ratios (extracted from the same live axe run, this session):

Element(s)	fg	bg	ratio	required
`.brand`	`#9d001e`	`#3c3f41`	1.43:1	4.5:1
`h1`	`#9d001e`	`#3c3f41`	1.62:1	3:1 (large text)
`p.tagline`	`#808080`	`#3c3f41`	2.68:1	4.5:1
Service links (×3)	`#a9b7c6`	`#4e5254`	3.86:1	4.5:1

These correspond to the theme's `--color-accent` (`#9d001e`) and `--color-text-secondary` (`#808080`) tokens against `--color-bg-secondary` (`#3c3f41`) / `--color-bg-tertiary` (`#4e5254`) — a real theme-contrast defect in the currently-active (Darcula-style) palette, not a one-off.

Per this task's scope (a new `tests/ux/` suite and/or `frontend/` **test** files only — no production Angular template/CSS edits were made), this assertion is left **failing on purpose**: it is the new automated gate correctly catching real product debt before a human QA pass would have (§11.4.238). **This is a release-relevant finding, not a suite defect** — route it to a tracked follow-up item for whoever owns `frontend/` theme tokens.

Honest gaps

- **`frontend/e2e/*.spec.ts` (the Angular dev-server-hosted app) was not scanned** — only the equivalent build served at `:7187` was, because starting `ng serve` was out of budget for this dispatch. Since `:7187` serves the same compiled Angular bundle, this is a reasonable proxy but not a substitute for scanning the dev-server-hosted route set directly (different routes, e.g. `/jackett/*`, were not scanned at all — this pass only covers the dashboard landing view). Command that would close this gap: `cd frontend && npm run start & npx playwright test` combined with pointing `tests/ux/test_live_dashboard_accessibility.py` (or a sibling) at `http://localhost:4200`.
- **Only the landing view was scanned live** — the Jackett credentials / indexers sub-routes, dialogs, and toast notifications were not driven and scanned. `axe-core`'s ruleset and the keyboard-nav helpers in `tests/ux/conftest.py` are reusable for those routes; adding `page.goto(f"{merge_service_live}/jackett")`-style tests is the natural next increment.
- **Environment gotcha, unrelated to this suite:** `python3 -m pytest` with default plugin autoloading currently fails to even collect *any* test in this repository (`ModuleNotFoundError: No module named 'rpds.rpds'`, transitively imported by the `schemathesis` pytest-plugin entry point via `jsonschema/referencing/rpds-py`). This is a broken native-extension install pre-existing in this environment, reproduced against an unrelated file (`tests/e2e/test_crossapp_theme.py --collect-only`) — not something introduced by this suite, and out of this

dispatch's scope to fix (`pyproject.toml` is not in scope). Every command in this document and this session's transcript was run with the workaround: `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest ... -p pytest_timeout`.

Running the suite

```
cd /path/to/boba
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest tests/ux/ -p
        pytest_timeout -v
```

(See "Honest gaps" above for why `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1` is currently required project-wide, not specific to this suite.)